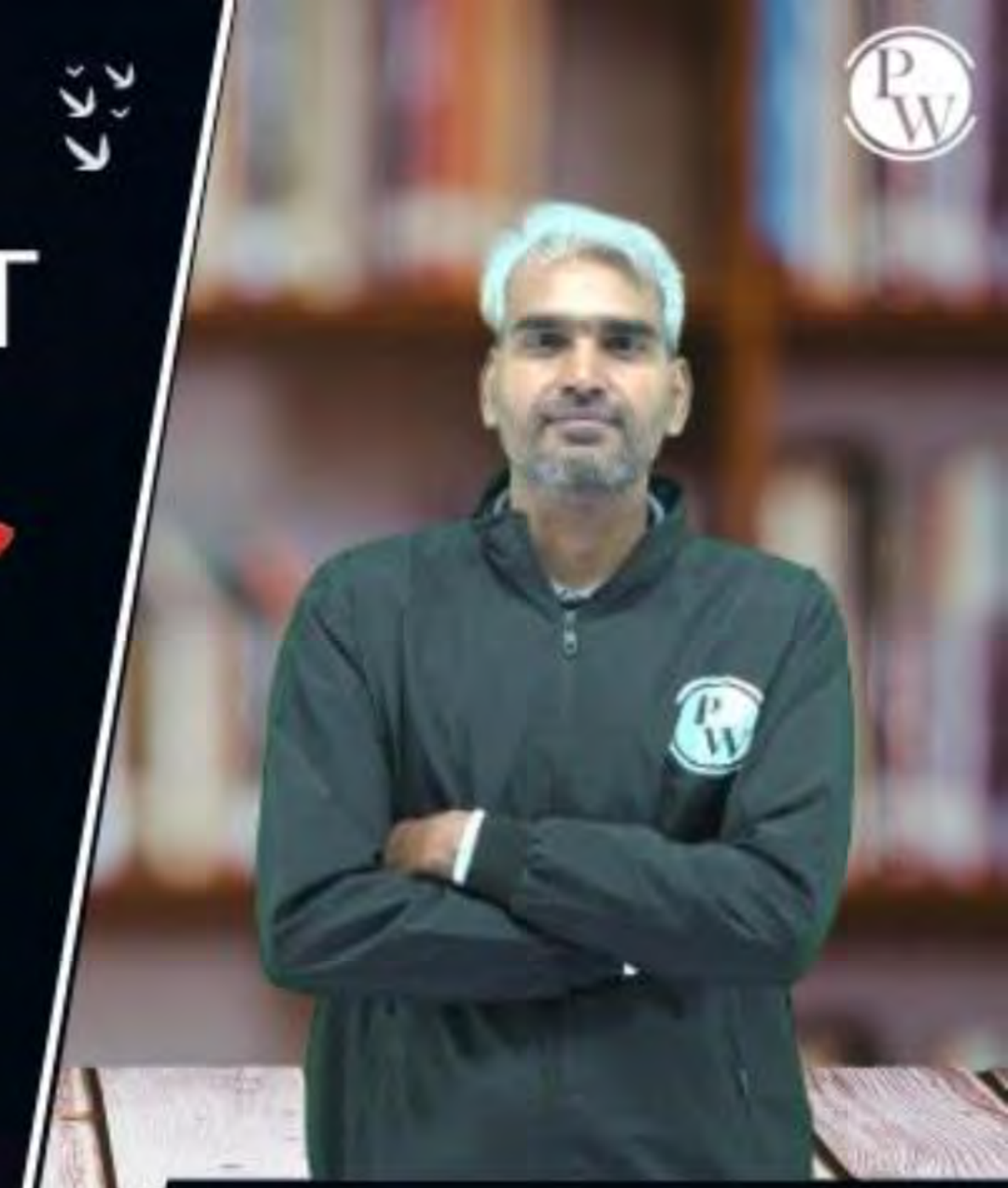
Computer Science & IT



Database Management System

File organization And Indexing

Lecture No. 03



By- Vishal Sir

Recap of Previous Lecture



- Topic: Search-key & Type of File
  - Ordered
  - Unordered
- Topic: IO cost without indexing — "N" or $\lceil \log_2 N \rceil$ (N = No. of disk blocks to store DB file)
- Topic: Index file {let uses M blocks}
- Topic: IO cost with indexing — $\lceil \log_2 M \rceil + 1$
- Topic: Categories of index
  - Dense
  - Sparse

Topics to be Covered



- Topic: Types of index
- Topic: Primary Index
- Topic: Clustering Index
- Topic: Secondary Index
- Topic: Multilevel Index

Slide



Topic : Database-File-Records

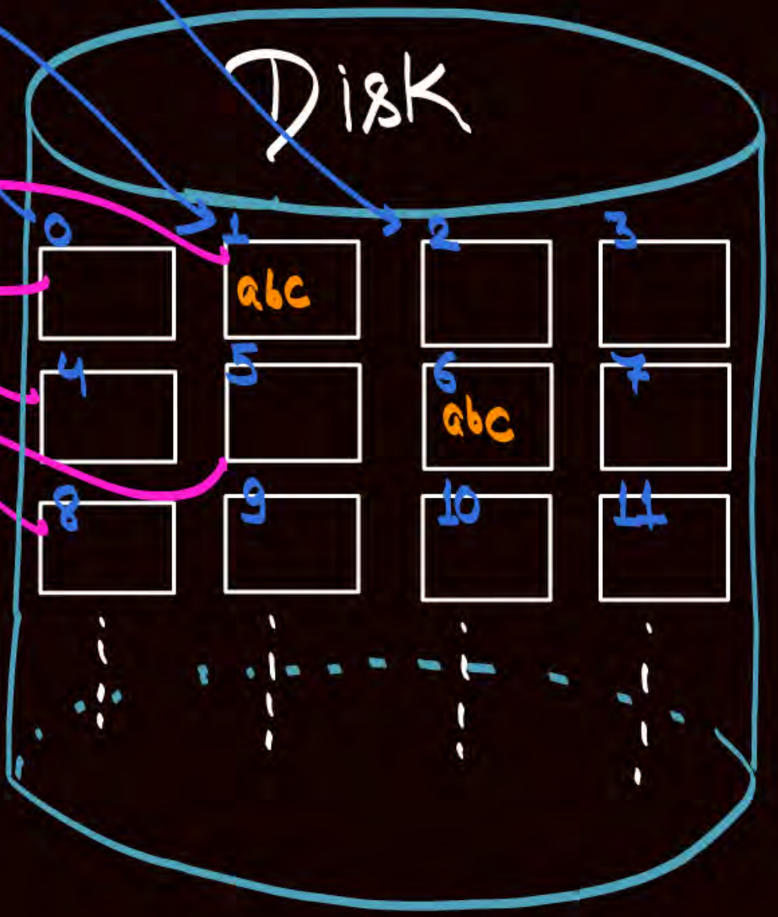
- Database is a collection of file.
- Files are used to store the records
- Disk blocks are used to store the records of the file.

  Records of different files can not be stored in the same block of the disk.



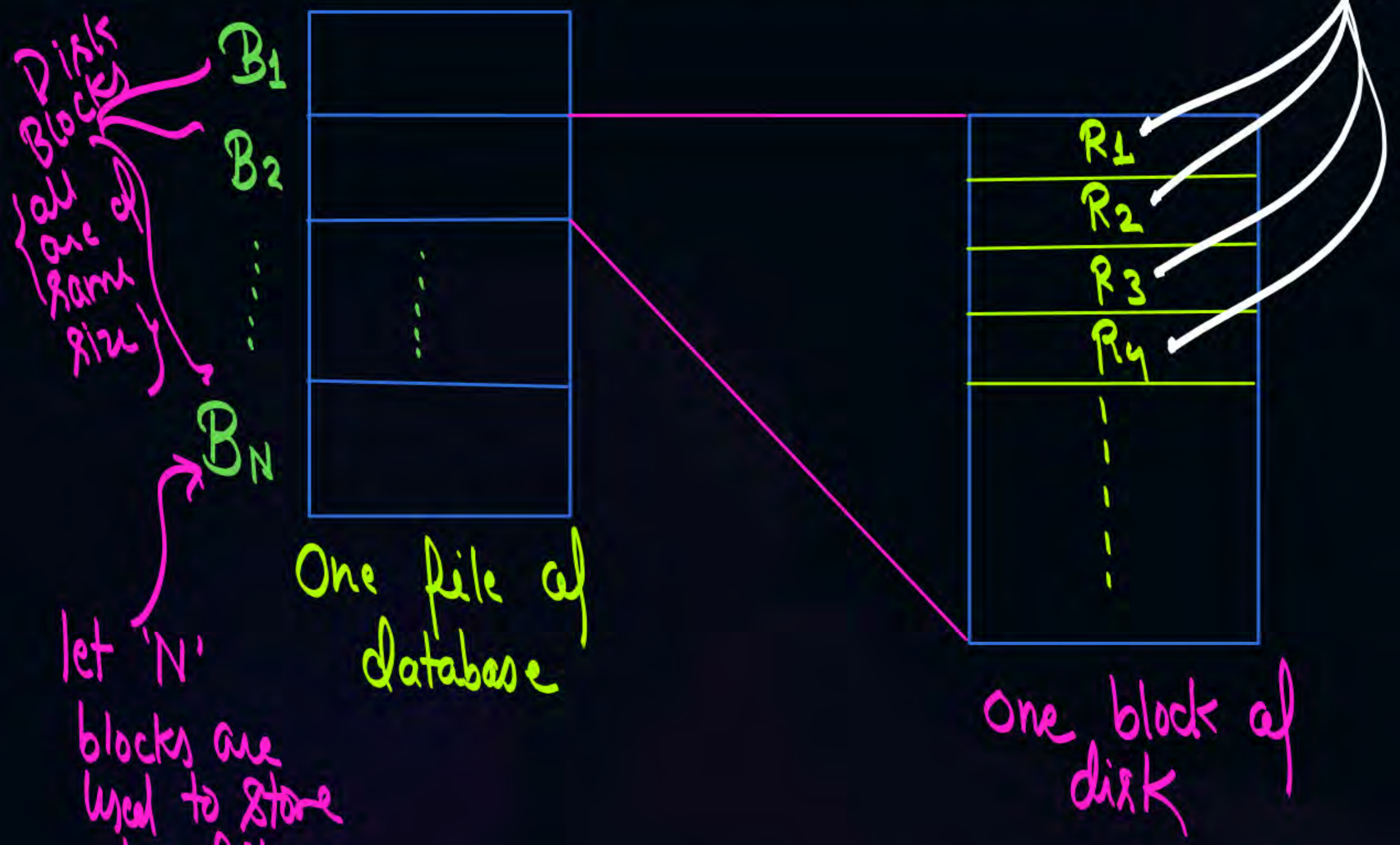
* Each block of the disk will be of Equal size
* Each block of disk is identified by a unique number known as "disk block address"
* One disk block can store multiple records of the same file (Not of different files)

Disk Blocks





Topic : Database-File-Records





Topic : Types of Records

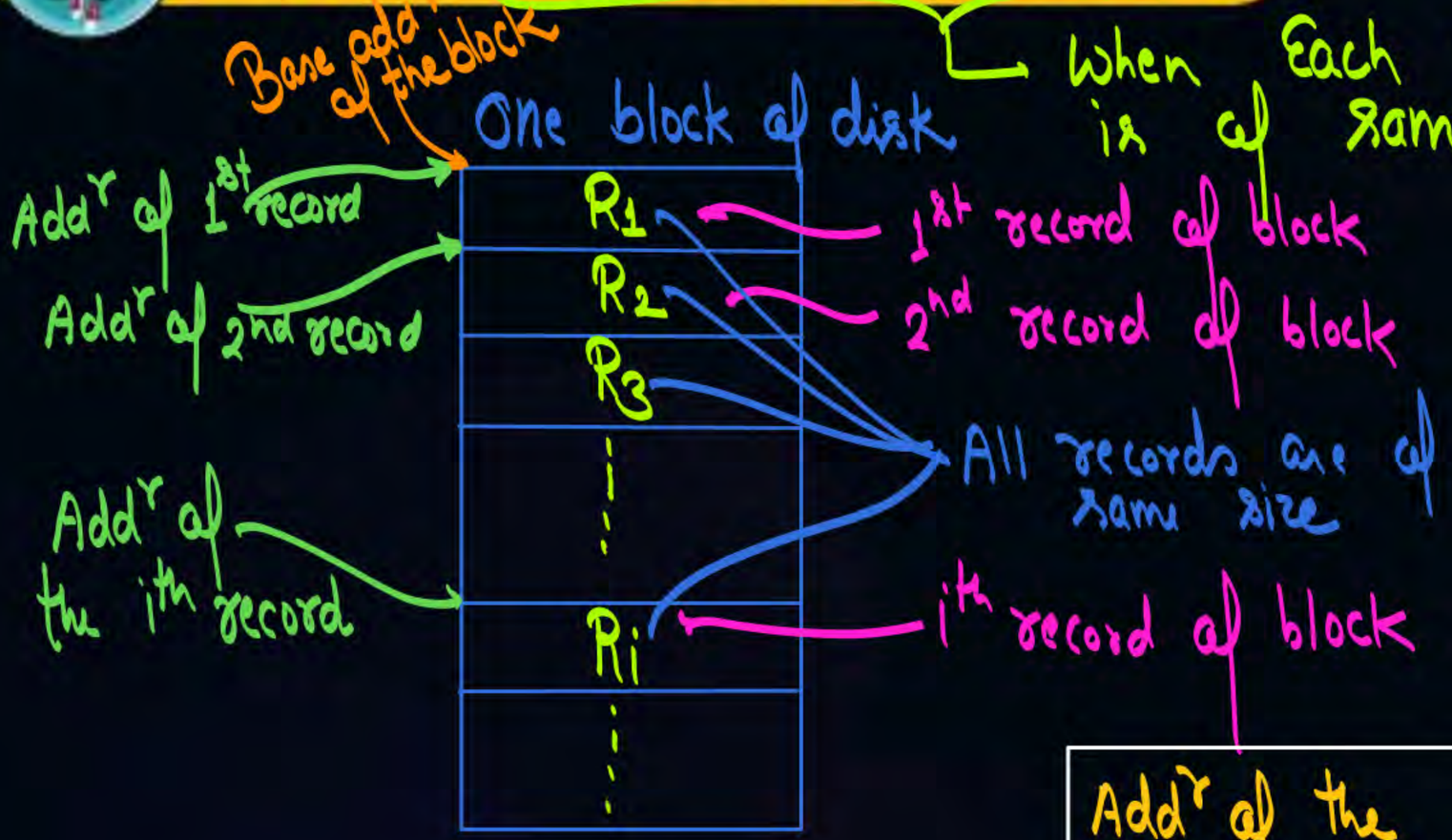
Two types of records are possible

(1) Fixed length records

(2) Variable length records.

Topic : Fixed length records

When Each record of the file is of same size

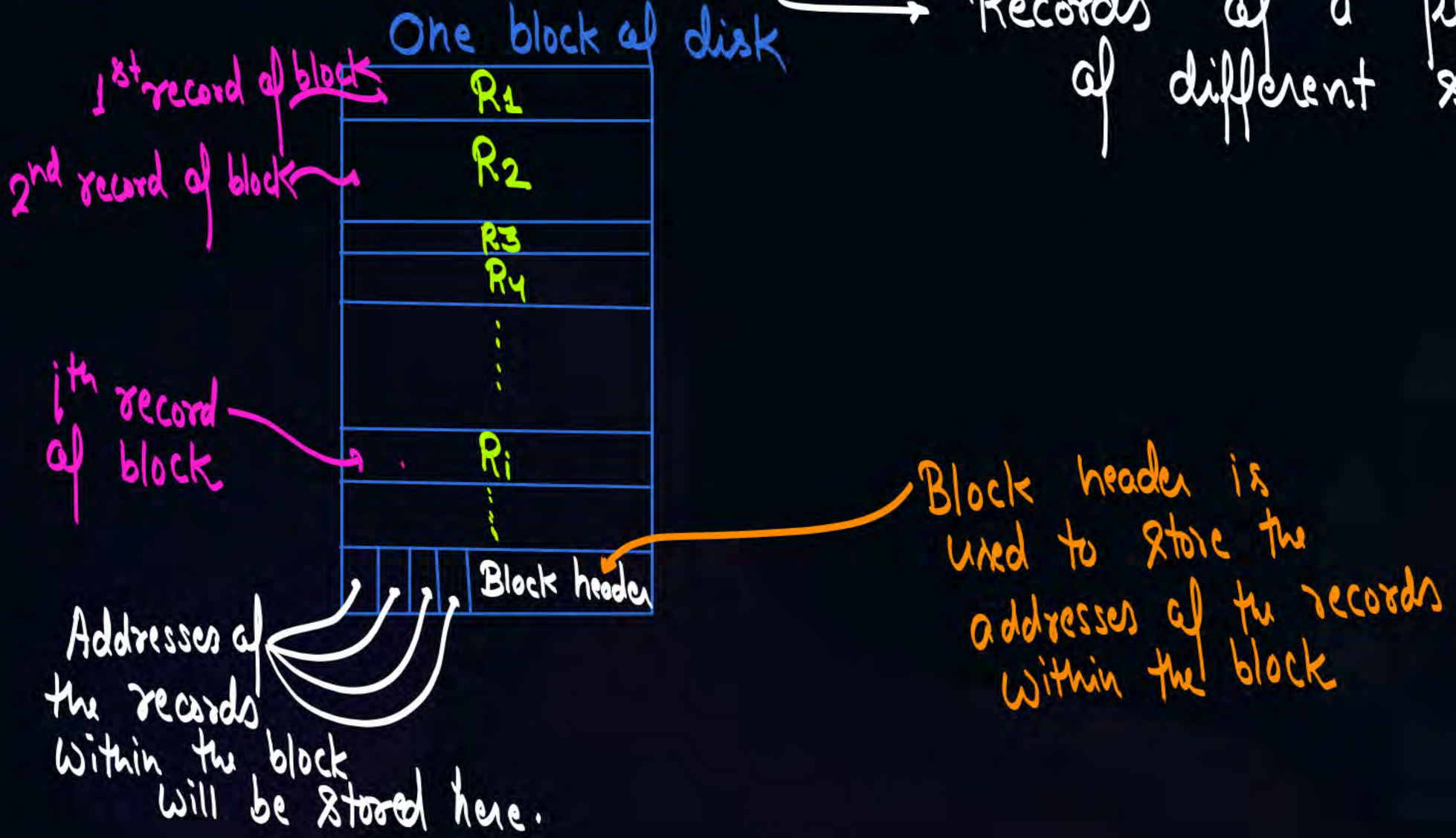


Addr of the ith record of a block = Base addr of the block + $(i-1)$ * Record size



Topic : Variable length records

→ Records of a file may be of different sizes.



Block heads may be required with fixed length records as well, in order to store the address of the next block used to store the records of the file { it will be required if "link allocation" is used to allocate the blocks of the disk }

Pointer used to provide the address of the next block of the file is called "Block anchor"

Note :- ① If nothing is specified in the question about "block header size" then we will consider it Zero (negligible) { i.e. Complete block size can be used to store the records of the file }.

② If "block header size" is specified in the question, then effective space available to store the records of the file = (Disk block size − Block header size)



Topic : Blocking Factor (Bf)

Blocking factor (Bf) of the block of the disk is defined as average number of records stored per block of disk

$$\text{Blocking factor} = \frac{\text{Block size}}{\text{Avg. record size}}$$ { when block header size is not given }

$$\text{Blocking factor} = \frac{\text{Block size} - \text{Block header size}}{\text{Avg record size}}$$ { When Block header size is given }



Topic : IO cost

IO Cost of an access can be defined as number of disk blocks that needs to be transferred from secondary memory to main memory

If no. of blocks transferred are more, then high IO Cost

If no. of blocks transferred are less, then low IO Cost



Topic : Organization of Records

There are two ways to organize records of the file in the blocks of disk

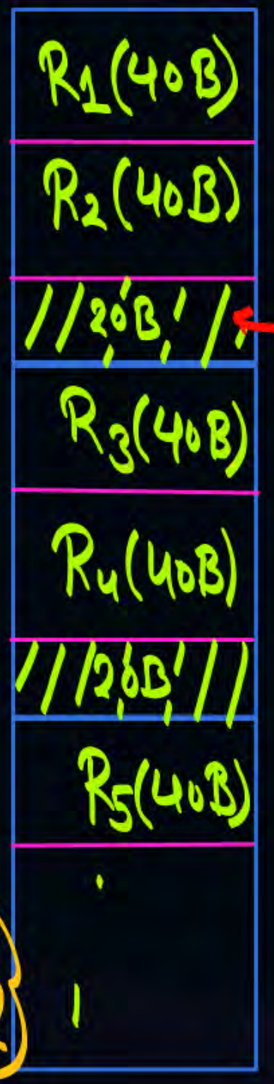
1. Unspanned Organization
2. Spanned organization

Topic : Un-spanned Organization

{ In unspanned organization a record must be stored Completely within a single block }

Let, Block size = 100 Bytes

Record size = 40 Bytes



B_1 = 100 Bytes

B_2 = 100 Bytes

B_3 = 100 Bytes

Available space is not sufficient to store a record Completely

↓

i. It will be wasted.

↓

it is called internal fragmentation

Blocking factor = 2

(using un-spanned organization)

Blocking factor (using unspanned orgn) = $\left\lfloor \frac{\text{Block size} - \text{Block header size}}{\text{Record size}} \right\rfloor = \left\lfloor \frac{100-0}{40} \right\rfloor = 2$

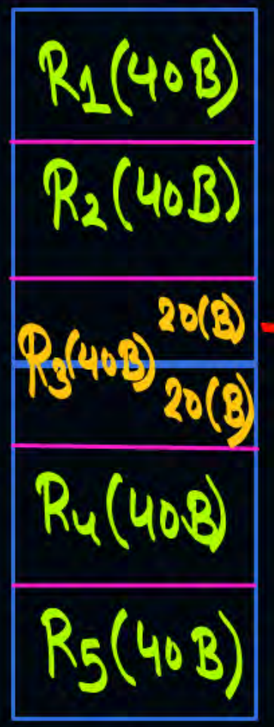


Topic : spanned Organization

{ In spanned organization a record is allowed to span in two blocks }

Let, Block size = 100 Bytes

Record size = 40 Bytes



B_1 = 100 Bytes

B_2 = 100 Bytes

Using spanned organization there will be no internal fragmentation

Blocking factor = 2.5 (using spanned organization)

Blocking factor (using spanned orgn) = $\frac{\text{Block size} - \text{Block header size}}{\text{Record size}} = \frac{100 - 0}{40} = 2.5$

Unspanned orgn vs Spanned orgn

Unspanned orgn	Spanned orgn
① Internal fragmentation	① No internal fragmentation
② Less IO Cost { one address is found, to access any record we need to transfer only one block from disk }	② More IO Cost { once address is found, in order to access some records of the file we may need to transfer two blocks from disk to M.M! }
③ Blocking factor is an integer value	③ Blocking factor may be in fraction
④ More suitable for fixed length records	④ More suitable for variable length records.



Topic : Search-key & Type of File

need not have unique values

* **Search key :→** An attribute/field used to search for a record in database file is called search key.

EMP (database file)

Eid	- . - . -	Dept. No.
E_3	Block B_1	1
E_1		2
E_7	Block B_2	1
E_2		4
E_5	Block B_3	2
E_6		3

Retrieve record of the Employee with Eid = 'E5'

↓

Search will be performed based on Eid

↓

then Search key = Eid

Retrieve records of the Employee with Dept. No. = 2

↓

Search will be Performed based on Dept. No.

↓

then Search key = Dept. No.



Topic : Search-key & Type of File

need not have unique values

* **Search key :→** An attribute/field used to search for a record in database file is called search key.

EMP (database file)

Eid	- . - . -	Dept. No.
E_1	Block B_1	2
E_2		4
E_3	Block B_2	1
E_5		2
E_6	Block B_3	3
E_7		1

Retrieve record of the Employee with Eid = 'E5'

↓

Search will be performed based on Eid

↓

then Search key = Eid

Retrieve records of the Employee with Dept. No. = 2

↓

Search will be Performed based on Dept. No.

↓

then Search key = Dept. No.



Topic : Search-key & Type of File

Types of File: There are two types of files,

1. **Ordered file** :- If the records of the file are ordered (sorted) based on search key
2. **Unordered file** : If the records of the file are not ordered based on search key.



Topic : Search-key & Type of File

need not have unique values

* **Search key :→** An attribute/field used to search for a record in database file is called search key.

EMP (database file)

Eid	⋯	Dept. No.
E_3	Block B_1	1
E_1		2
E_7	Block B_2	1
E_2		4
E_5	Block B_3	2
E_6		3

let Search key = Eid

Records are unordered based on search key

∴ Unordered file w.r.t. Eid

let Seach key = Dept. No.

↓

Records are not ordered based on search key.

∴ Unordered file w.r.t. Dept. No. as well.



Topic : Search-key & Type of File

need not have unique values

* **Search key :→** An attribute/field used to search for a record in database file is called search key.

EMP (database file)

Eid	- . . . -	Dept. No.
E_1	Block B_1	2
E_2		4
E_3	Block B_2	1
E_5		2
E_6	Block B_3	3
E_7		1

let Search key = Eid

Records are ordered based on search key.

∴ Ordered file wrt Eid

Unordered w.r.t- Dept. No.



Topic : IO cost without indexing

IO cost → No. of blocks that needs to be transferred from sec. mem. to main memory in order to access a record.

Let "N" is the total number of disk blocks required to store a database file.

(i) Worst case IO cost = N { Linear Search }
{ w.r.t. Unordered file }

(ii) Worst case IO cost = $\lceil \log_2 N \rceil$ { If file is ordered, then Binary Search can be used }
{ w.r.t. Ordered file }

Ordered file vs Unordered file {Also known as Heap file}

Ordered file	Unordered file {Also known as Heap file}
① Binary Search is possible	① Binary Search is not possible
② Insertion of a new record will be a costly operation {As we need to maintain the Sorted order}	② Insertion will be easy {we can simply insert the new record at the end of the file}



Topic : Index file

Index file is used to reduce the IO Cost.

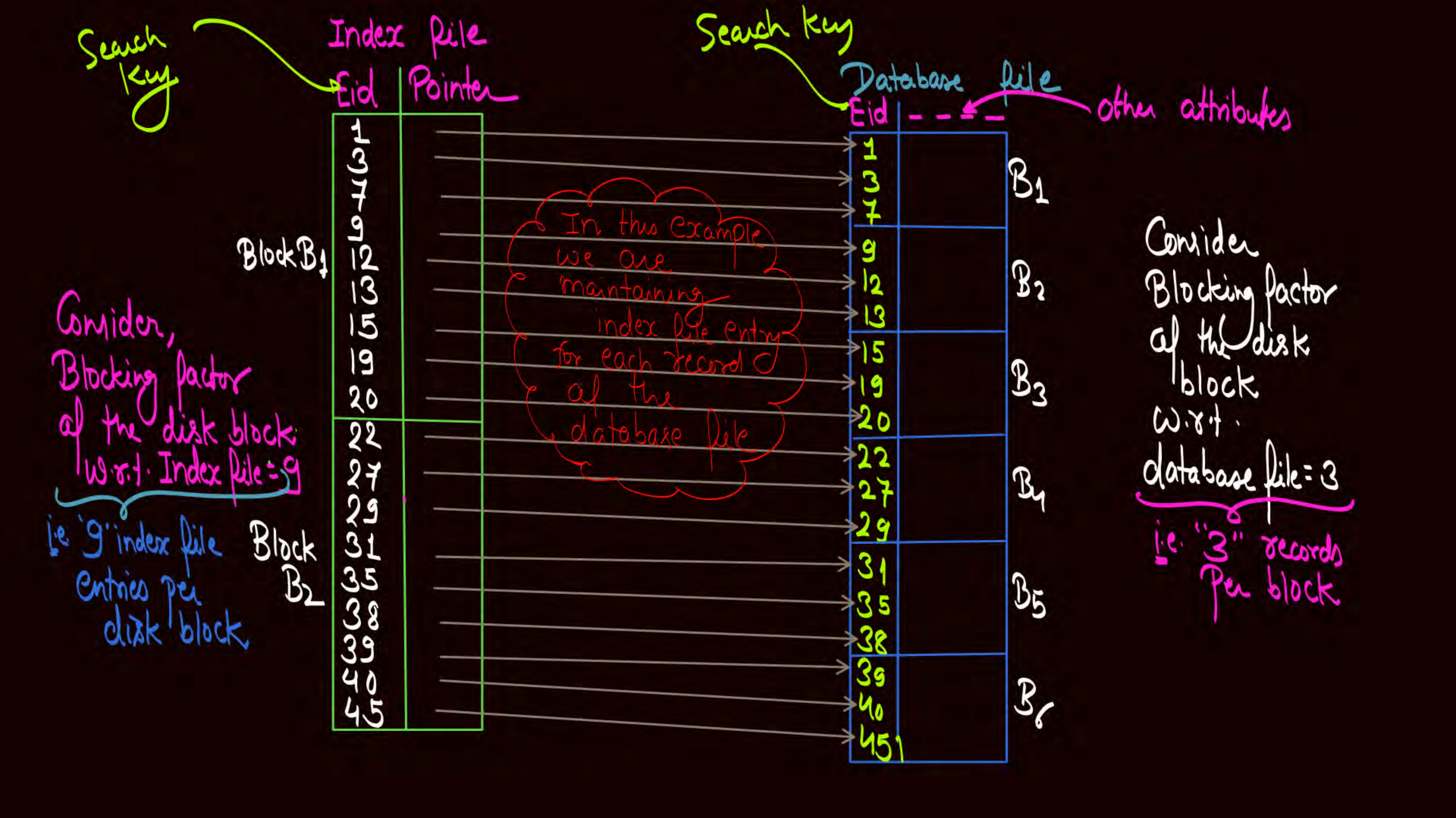
Each entry in the index file stores two information

1. Search key attribute value
2. Pointer pointing to the record / disk block corresponding to the search key value

Note:- Index file is stored in the blocks of disk.

Note:- Disk block size is fixed

Note: Size of disk block used = Size of disk blocks used to store the entries of the index file / Size of disk blocks used to store the records of database file


Search key
Index file
Eid
Pointer
Block B1
Block B2
1
3
7
9
12
13
15
19
20
22
27
29
31
35
38
39
40
45
In this example we are maintaining index file entry for each record of the database file
Search key
Database file
Eid
other attributes
B1
B2
B3
B4
B5
B6
Consider, Blocking factor of the disk block w.r.t. Index file = 9
i.e. "9" index file entries per disk block
Consider Blocking factor of the disk block w.r.t. database file = 3
i.e. "3" records per block

Note:-

① Index file entry size = (Search key field size + Pointer size)

② Index file entry = < Search key value, Pointer >

③ Database file Record size = Summation of size required to store each attribute of file {generally it is given in question}

④ Disk block size is same for database file as well as for index file

⑤ Blocking factor of Index block = $\frac{\text{Disk Block Size}}{\text{Index file entry size}}$ {i.e. wrt index file}

⑥ Blocking factor of database file block = $\frac{\text{Disk Block Size}}{\text{Record Size}}$ {i.e. wrt database file}

Note:

In general,

① Index file entry size << Database file Record size

and ② No. of entries in the index file <= No. of records in DB file

w.r.t. Sparse Index

w.r.t Dense Index

⑦ Blocking factor of index block > Blocking factor of Database block

$$\frac{\text{Block size}}{\text{entry size}} > \frac{\text{Block size}}{\text{Record size}}$$

$$\text{No. of blocks required to store index file} = \left\lceil \frac{\text{Total no. of entries in Index file}}{\text{Blocking factor of index Block}} \right\rceil$$

$$\text{No. of blocks required to store the database file} = \left\lceil \frac{\text{Total number of Records in DB file}}{\text{Blocking factor of database block}} \right\rceil$$

(8) Number of blocks required to store the index file $<<$ Number of blocks required to store the database file.

$$\frac{\text{Total no. of entries in Index file}}{\text{Blocking factor of index Block}} \leq \frac{\text{Total number of Records in DB file}}{\text{Blocking factor of database block}}$$

Note: In general, Index file will be ordered based on search key

→ i.e., to search for a key in the index file we can use binary search



Topic : IO cost with indexing

Let 'M' is the number of disk blocks required to store the index file. { We know, M << N }

N → Number of disk blocks required to store database file.

$$\text{Worst Case IO Cost (with Indexing)} = \lceil \log_2 M \rceil + 1$$

$\lceil \log_2 M \rceil$ → No. of blocks transferred to search for an entry in the index file

Binary search is possible

1 → From index file we will get the addr of actual record in the database file to access the records database block will also be transferred.



Topic : Categories of Index

There are two ways in which index file can be created.

① Dense Index file :

For each record of the database file if we maintain an entry in the index file, then it is called dense index file

i.e; No. of entries in the index file = No. of records in the DB file

② Sparse index file :

For a collection of records in the database file if we maintain a single entry in the index file, then it is called sparse index file.

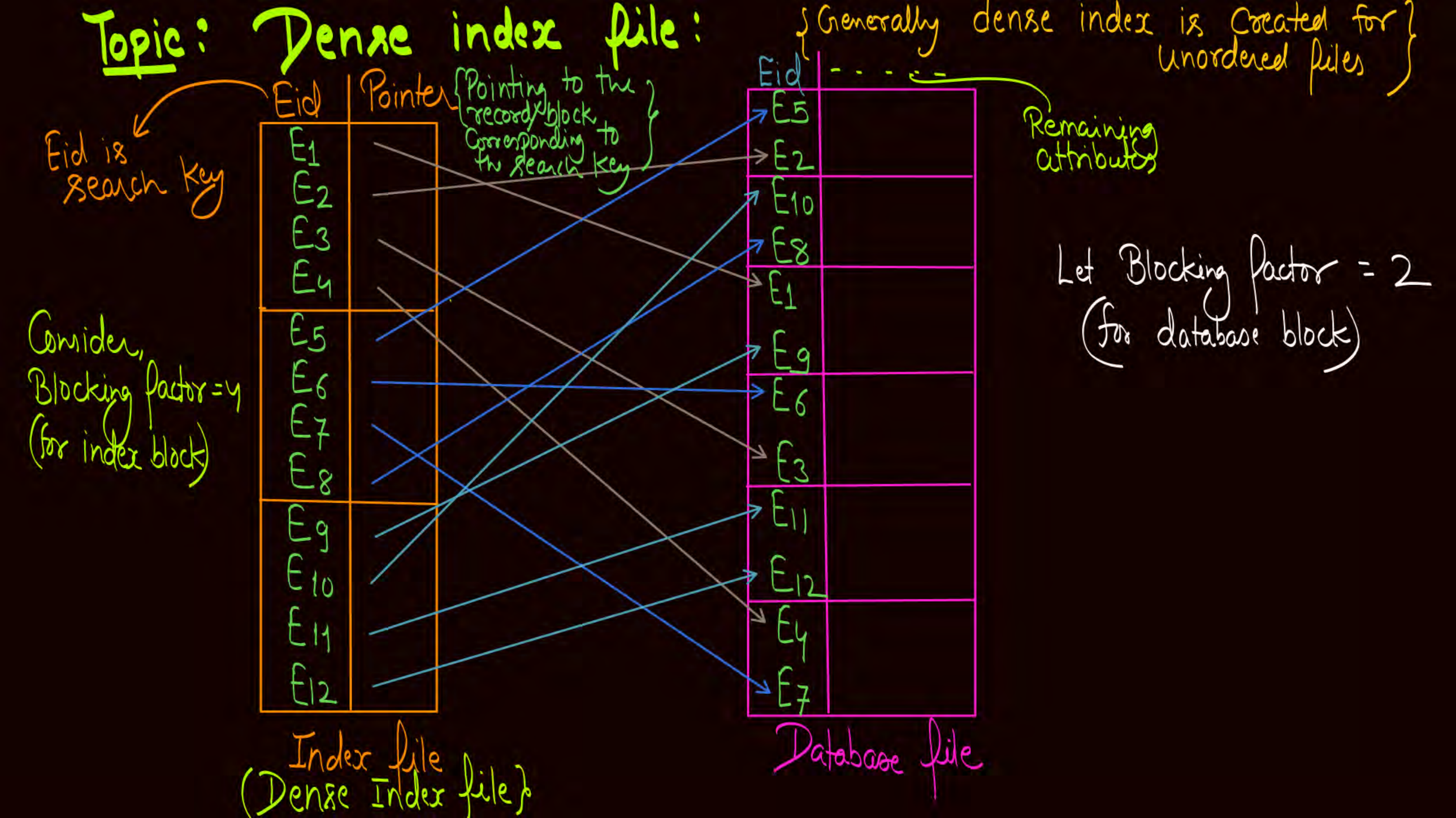
i.e; No. of entries in index file < No. of entries in the database file

Note: If index is sparse index, and nothing else is specified in the question about number of entries in the sparse index file, the by default,

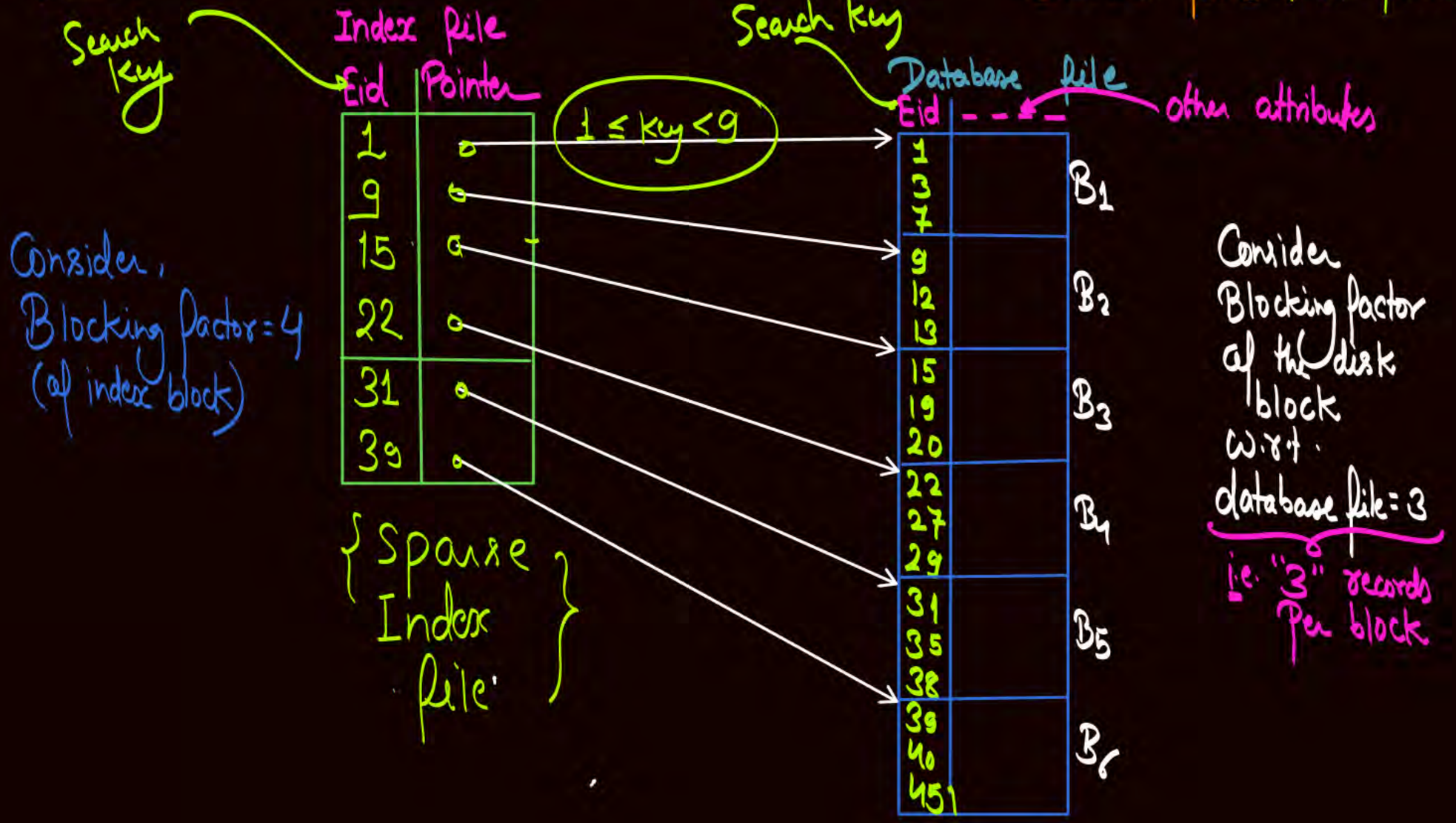
> No. of entries in sparse index file = No. of blocks required to store the database file
>
> i.e; One index file entry per database block

Topic: Dense index file:

{Generally dense index is created for unordered files}



Topic: Sparse index file :- {Generally Sparse index is created for Ordered file (not possible for unordered file)}



Note: ① If file is ordered based on search key, then both Sparse and dense indexes are Possible. {Generally Sparse will be more beneficial}

② If file is unordered based on search key then only dense index can be Created. {Sparse index can never be Created for unordered file}

Q :- Consider the following specifications

Disk block size = 1000 Bytes

Database record size = 100 Bytes

Search key attribute size = 12 Bytes

Block pointer size = 8 Bytes

Let total no. of records in DB file = 10,000

then,

(i) How many disk blocks are required for dense index file

(ii) How many disk blocks are required for sparse index file

(i) No. of disk blocks required (for dense index) = Total no. of entries in dense index file / No. of index entries per block

= No. of records in DB file / Blocking factor of index block

$$= \frac{10,000}{\left(\frac{\text{Block size}}{\text{index entry size}}\right)} = \frac{10,000}{\left(\frac{\text{Block size}}{\text{Search key size} + \text{Block ptr size}}\right)}$$

$$= \frac{10,000}{\left(\frac{1000}{12+8}\right)} = \frac{10,000}{\left(\frac{1000}{20}\right)} = \frac{10,000}{50} = 200$$

(ii) No. of disk blocks required (for sparse index) = Total no. of entries in sparse index file / No. of index entries per block

because 1 entry per database block

= No. of disk blocks required to store DB file / Blocking factor of index file

No. of disk blocks required to store DB file = Total no. of records / No. of records per block

= 10,000 / (Block size / Record size) = 10,000 / (1000 / 100)

= 10,000 / 10 = 1000

= 1000 / (1000 / (12+8)) = 1000 / 50

= 20



Topic : Types of Index

- Single level Index
  - Primary Index
  - Clustering index
  - Secondary index
- Multi-level index
  - Static Multi level Index
  - Dynamic Multi level Index
    - B tree
    - B+ tree

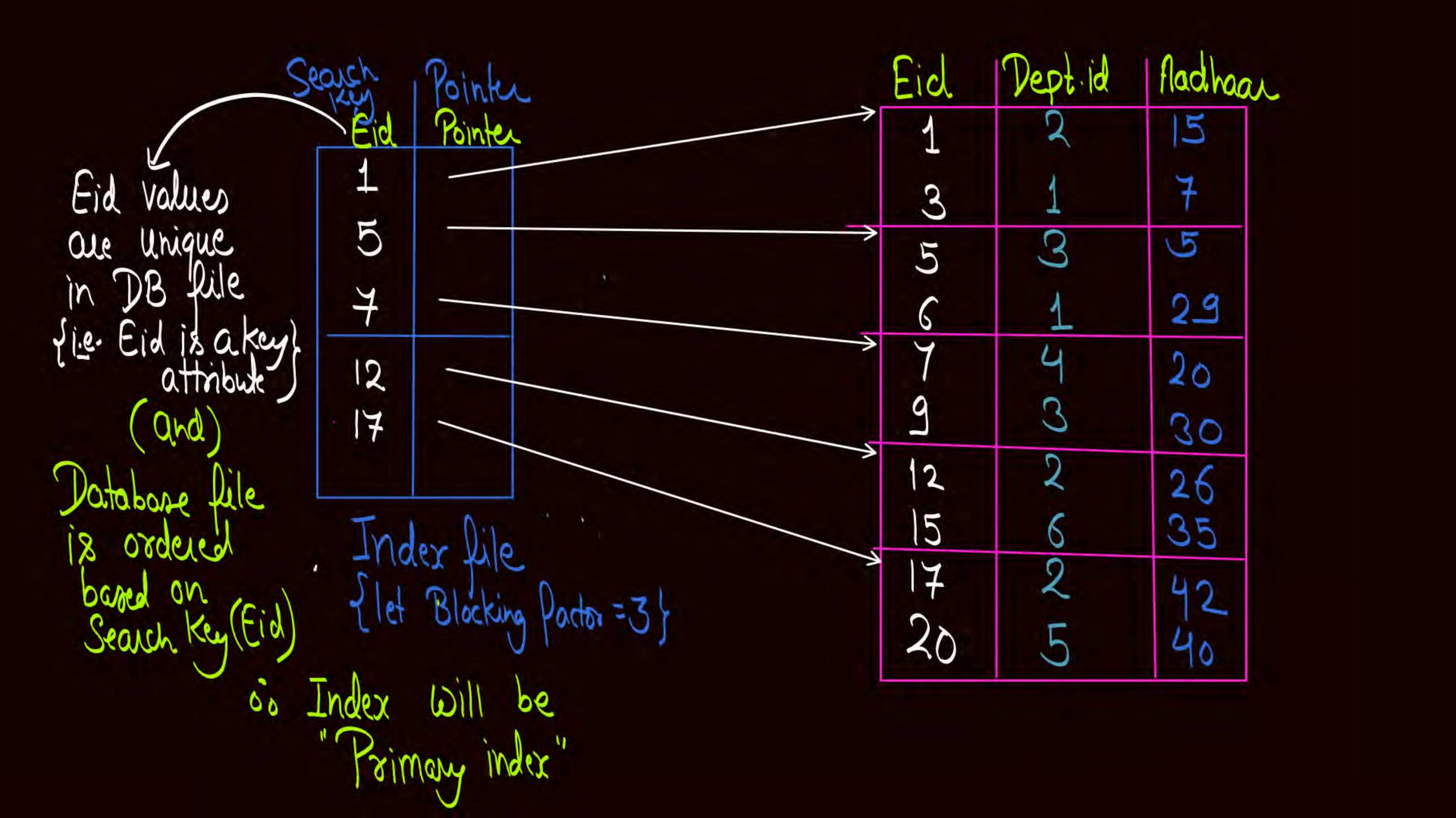


Topic : Primary Index

If ① Search key attribute is a key attribute (i.e. unique values)

② File is ordered based on search key

then Index is called primary index.


Search key
Eid
Pointer
Pointer
Eid values are unique in DB file {i.e. Eid is a key attribute}
(and)
Database file is ordered based on Search key (Eid)
Index file
{let Blocking Factor = 3}
∴ Index will be "Primary index"
Eid
Dept.id
Aadhaar
1
3
5
6
7
9
12
15
17
20
2
1
3
1
4
3
2
6
2
5
15
7
5
29
20
30
26
35
42
40



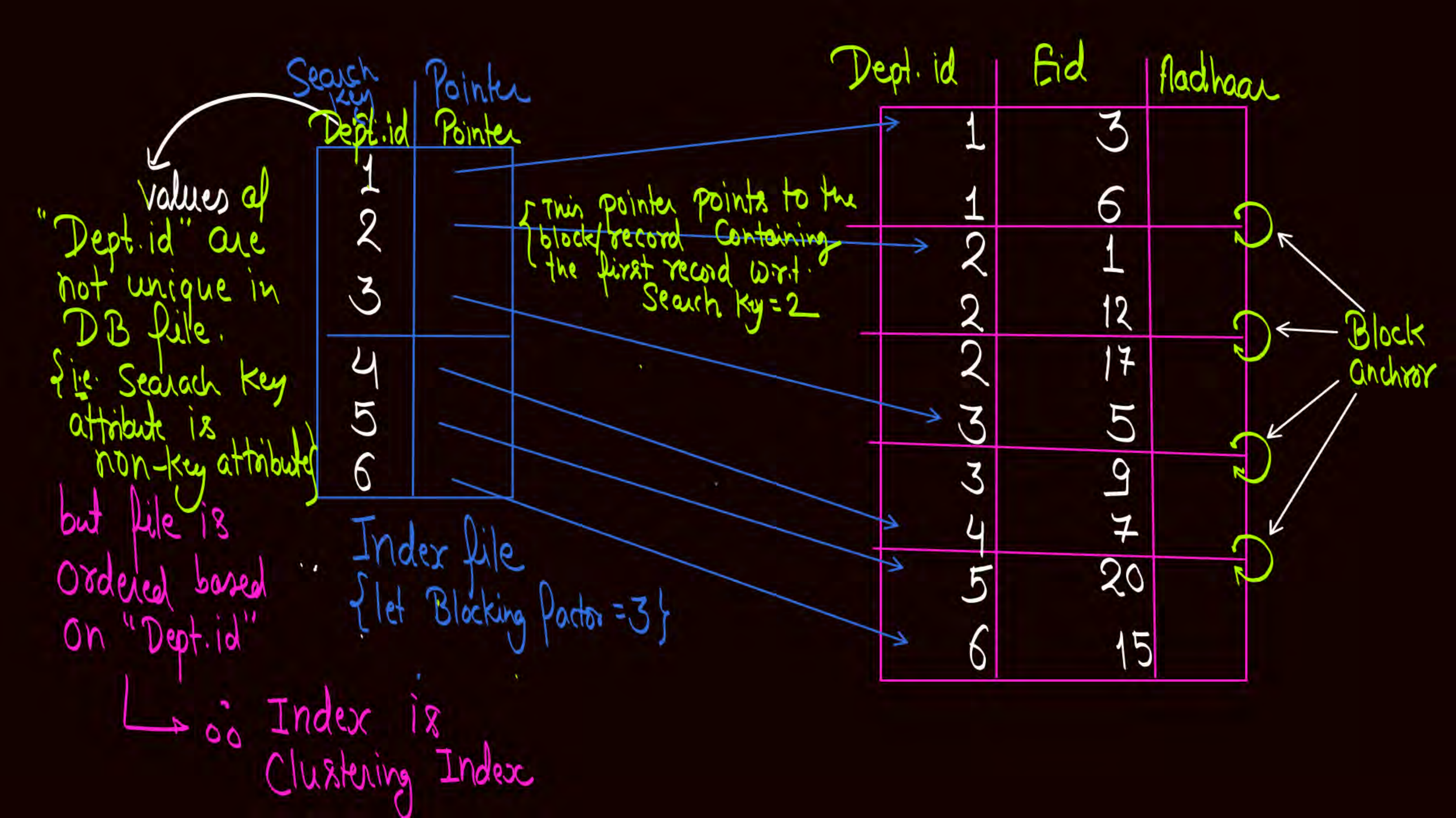
Topic : Clustering Index

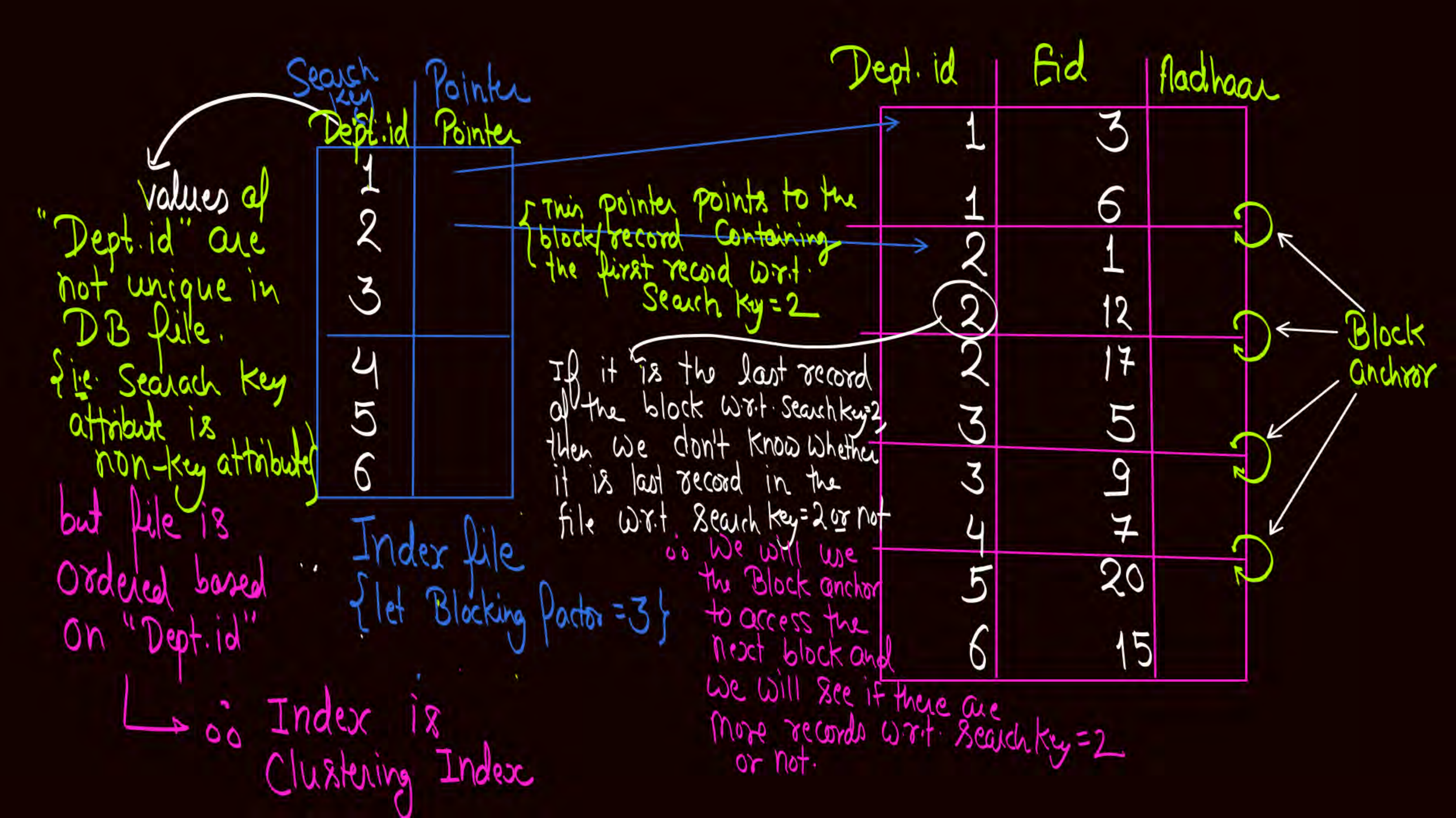
i.e. Values are not unique

If ① Search key attribute is a non-key attribute

② file is ordered based on Search key

then Index is called Clustering index


Search key
Dept.id
Pointer
Pointer
Values of "Dept.id" are not unique in DB file.
{i.e. Search key attribute is non-key attribute}
but file is ordered based on "Dept.id"
∴ Index is Clustering Index
This pointer points to the block/record containing the first record w.r.t. Search Key = 2
Index file
{let Blocking Factor = 3}
Dept. id
Eid
Aadhaar
Block anchror


Search key
Dept.id
Pointer
Pointer
Values of "Dept.id" are not unique in DB file.
{i.e. Search key attribute is non-key attribute}
but file is ordered based on "Dept.id"
∴ Index is Clustering Index
Index file
{let Blocking Factor = 3}
This pointer points to the block/record containing the first record w.r.t. Search key = 2
If it is the last record of the block w.r.t. Search key=2, then we don't know whether it is last record in the file w.r.t. Search key = 2 or not
∴ We will use the Block anchor to access the next block and we will see if there are more records w.r.t. Search key = 2 or not.
Dept. id
Eid
Aadhaar
Block anchor

Note:

① Primary index may be a Sparse index or dense index
(generally Primary index is Sparse index)

② Clustering index is always a sparse index.

③ At most one primary index can be Created on a database file {because file must be ordered based on search key}

④ At most one Clustering index can be Created on a database file {because file must be ordered}

Note:

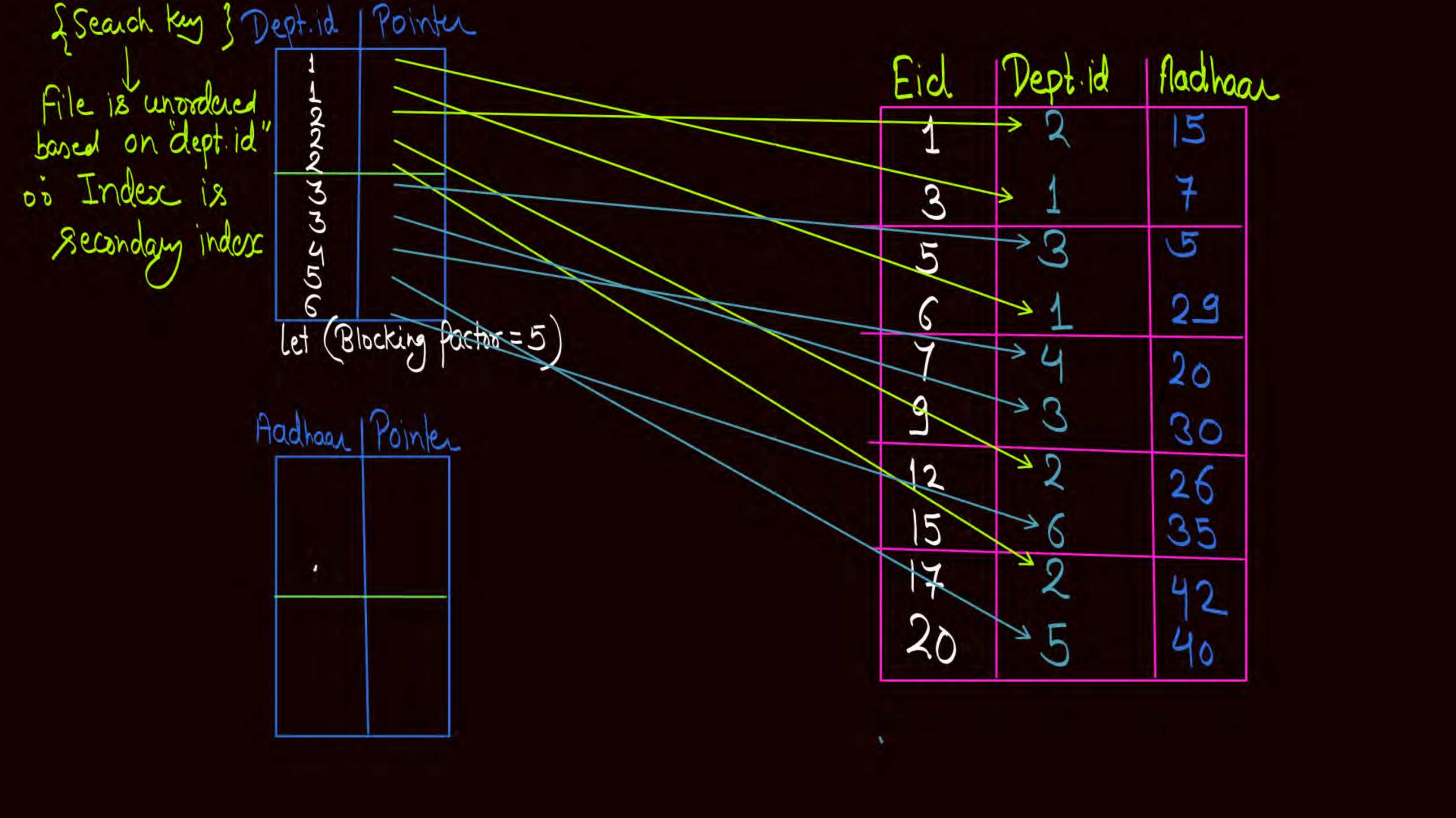
⑤ On a database file we can either have a primary index or a clustering index but not both
{Because file must be ordered}

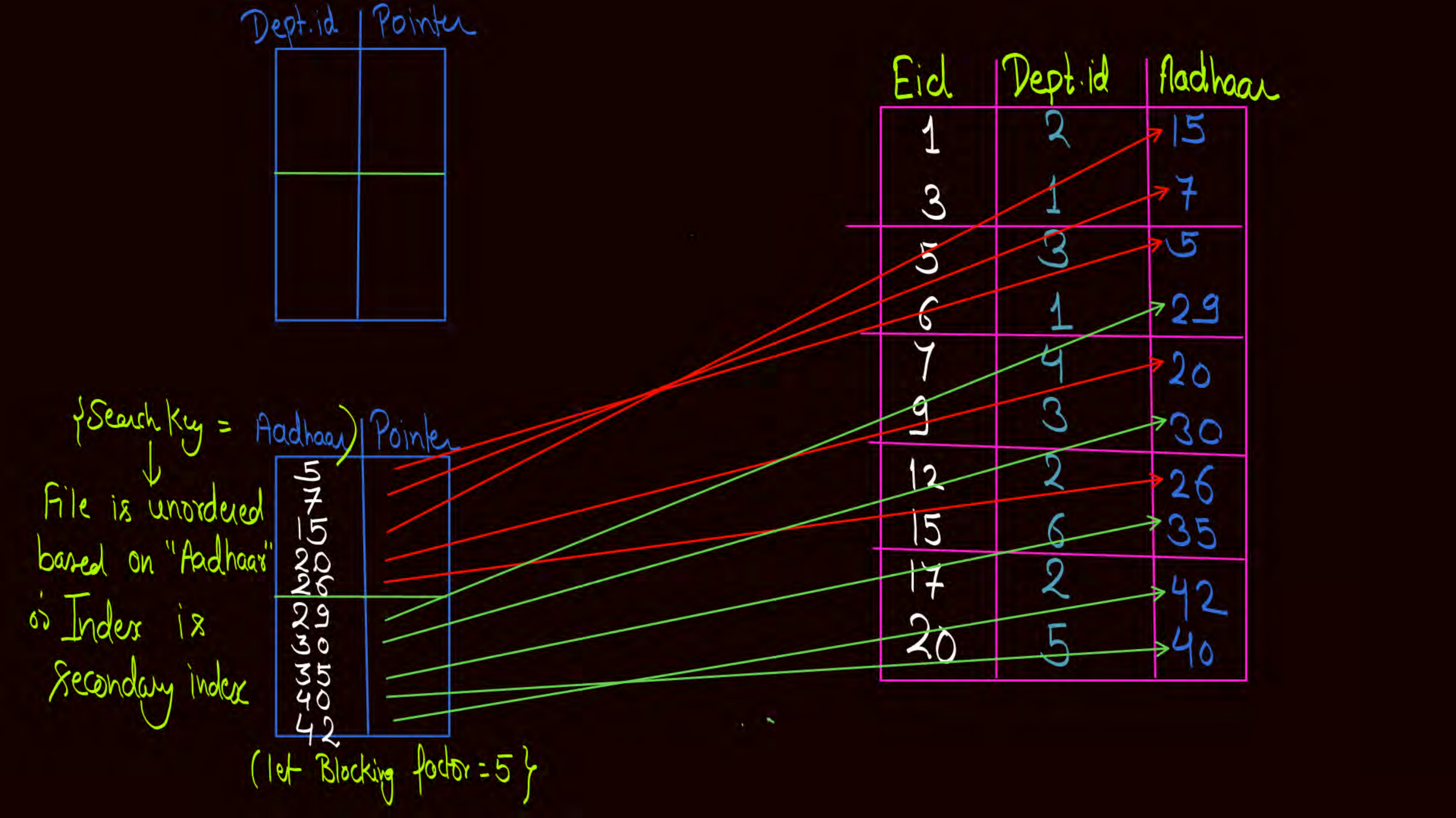


Topic : Secondary Index

If file is unordered based on search key attribute then Index is called secondary index

Search key attribute may be a key attribute or May be a non-key attribute

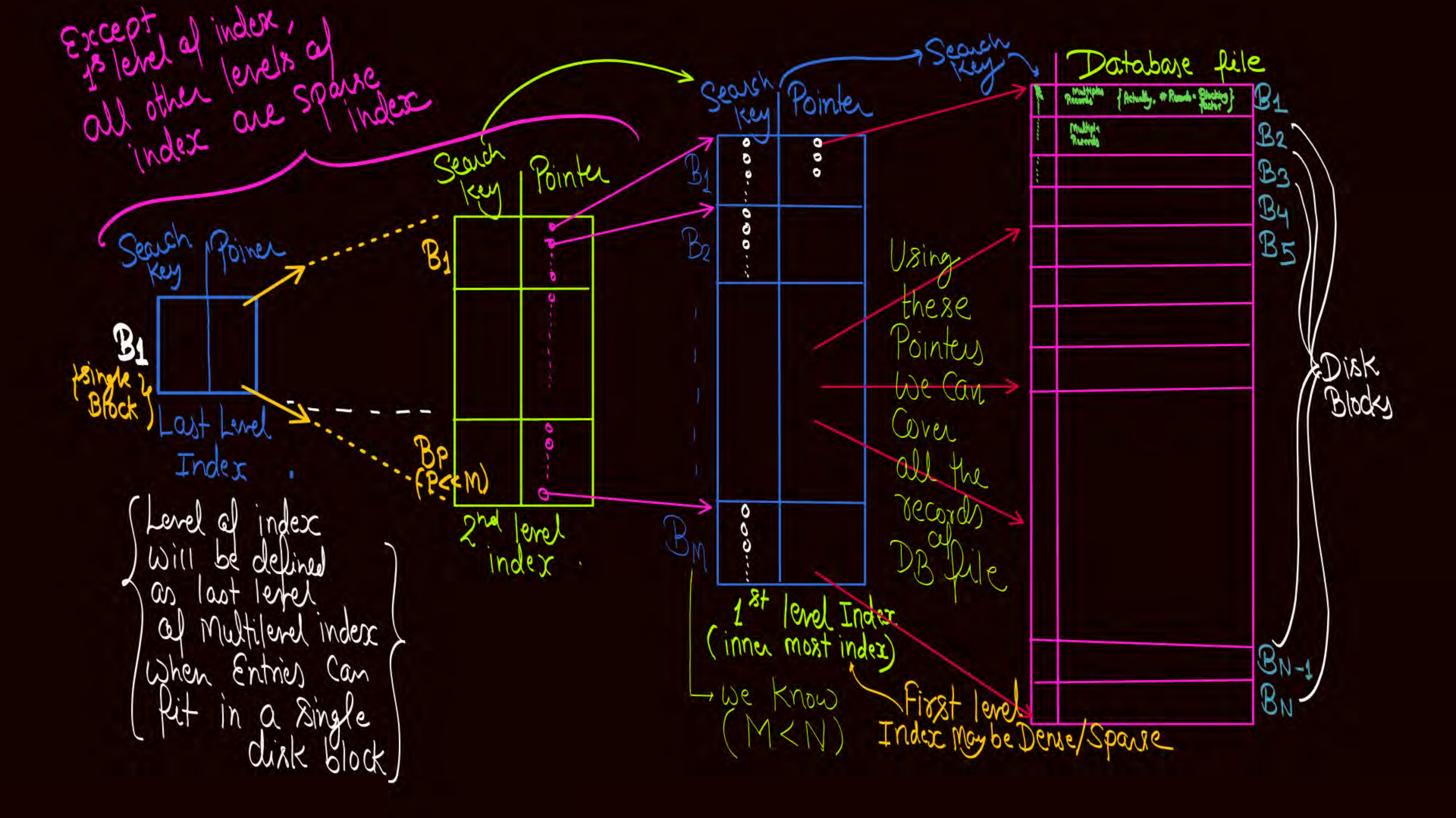

{Search key} Dept.id | Pointer
File is unordered based on "dept.id"
∴ Index is secondary index
1
1
2
2
3
3
5
5
6
let (Blocking factor = 5)
Aadhaar | Pointer
Eid | Dept.id | Aadhaar
1 | 2 | 15
3 | 1 | 7
5 | 3 | 5
6 | 1 | 29
7 | 4 | 20
9 | 3 | 30
12 | 2 | 26
15 | 6 | 35
17 | 2 | 42
20 | 5 | 40


Dept.id
Pointer
Eid
Dept.id
Aadhaar
1
2
15
3
1
7
5
3
5
6
1
29
7
4
20
9
3
30
12
2
26
15
6
35
17
2
42
20
5
40
{Search Key = Aadhaar}
Pointer
File is unordered based on "Aadhaar"
∴ Index is Secondary index
5
7
15
22
20
26
32
29
30
33
35
40
42
(let Blocking factor = 5)

Note: If database file is unordered w.r.t. search key {i.e. secondary index}, then our first objective will be to provide an ordering w.r.t. search key.

∴ Secondary index is always a dense index.

Note:- Any number of secondary index are possible on database file {Because file don't need to be ordered based on Search key}


Except 1st level of index, all other levels of index are sparse index
Search Key
Pointer
B1
Last Level Index
(single Block)
Bp (P<=M)
2nd level index
BM
1st level Index (inner most index)
we know (M<N)
Using these Pointers we can Cover all the records of DB file
Database file
Multiple Records
{Actually, # Records = Blocking factor}
B1
B2
B3
B4
B5
BN-1
BN
Disk Blocks
First level Index may be Dense/Sparse
Level of index will be defined as last level of Multilevel index when Entries can fit in a single disk block

* Multi-level index can improve significantly in terms of IO Cost.

* **IO Cost with Multi-level Index :-**

In a multi-level index if there are "S" levels of index then,

IO Cost (to access a record) = No. of levels in Multi-level index + 1

- No. of levels in Multi-level index: from each level of index 1 block will be transferred
- + 1: This is w.r.t. block of the database file to access the record based on addr provided by inner most level of index

IO Cost = S + 1

Using multi-level index

IO Cost = No. of levels + 1

for all accesses.

{i.e. Best case, worst case, Avg case always same}

In multi-level Index :-

1. 1st level of index can be dense / sparse and other levels of index are always sparse.
2. IO Cost = No. of levels + 1

Note: * The multi-level index that we have just discussed is suitable only for static database
{i.e. insert/delete/update op^n are very rare}

* If database is dynamic {i.e. insert/delete/update are frequent} then after every such operation we will have to restructure all the levels of multi-level index; and it will be time consuming

i.e. if database is dynamic then significant portion of time will be wasted in maintaining the index itself.

* Above multi-level index is not suitable for Dynamic database,

∴ We use dynamic multilevel index tree for dynamic database

- B tree
- B+ tree

} These are self-balancing multi-way search tree.



2 mins Summary

- Topic: Types of index
- Topic: Primary Index
- Topic: Clustering Index
- Topic: Secondary Index
- Topic: Multilevel Index

Slide

THANK - YOU

Slide